Document Information	
Filename	TP_F101U-00030_User_Guide_Security
Status	Release
Revision Date	Jul 3,2024
Version	06.00
Document ID	TP_F101U-00030

Standard Proprietary Terms

THE REPRODUCTION, DISTRIBUTION AND UTILIZATION OF THIS DOCUMENT AS WELL AS THE COMMUNICATION OF ITS CONTENTS TO OTHERS WITHOUT EXPRESS AUTHORIZATION IS PROHIBITED. OFFENDERS WILL BE HELD LIABLE FOR THE PAYMENT OF DAMAGES. ALL RIGHTS RESERVED IN THE EVENT OF THE GRANT OF A PATENT, UTILITY MODEL OR DESIGN.

© Continental Automotive GmbH 2019
Confidential and Proprietary

Sensitivity: Internal & Restricted

Table of contents

1 Introduction 4
1.1 Notational Conventions 4
2 Setup 4
2.1 Tools/Equipment Needed 4
2.2 Setup Instructions 4
2.3 Setup Troubleshooting 4
3 Features/Components 4
3.1 How to setup password through persistency 4
3.1.1 Overview 4
3.1.1.1 Platform Content Overview 4
3.1.1.2 Product Expectations 5
3.1.1.3 Extending Platform Functionality 7
3.1.2 Setup 7
3.1.2.1 Tools/Equipment Needed 7
3.1.2.2 Setup Instructions 7
3.1.2.3 Setup Troubleshooting 7
3.1.3 Usage 7
3.1.3.1 Basic Instructions 7
3.1.3.2 Detailed Instructions 7
3.1.3.3 Testing/Error Reporting 7
3.2 How to sandbox an application 7
3.2.1 Overview 7
3.2.1.1 Platform Content Overview 7
3.2.1.2 Product Expectations 8
3.2.1.3 Extending Platform Functionality 8
3.2.2 Setup 8
3.2.2.1 Tools/Equipment Needed 8
3.2.2.2 Setup Instructions 8
3.2.2.3 Setup Troubleshooting 9
3.2.3 Usage 9
3.2.3.1 Basic Instructions 9
3.2.3.2 Detailed Instructions 9
3.2.3.3 Testing/Error Reporting 9
3.3 OpenSSL CryptoProvider 10
3.3.1 Overview 10
3.3.1.1 Platform Content Overview 11
3.3.1.2 Product Expectations 13
3.3.1.3 Extending Platform Functionality 13
3.3.2 Setup 13

© Continental Automotive GmbH 2019
Confidential and Proprietary

Sensitivity: Internal & Restricted

3.3.2.1 Tools/Equipment Needed 14
3.3.2.2 Setup PSK Key Instruction 14
3.3.2.3 Setup Troubleshooting 15
3.3.3 Usage 15
3.3.3.1 Basic Instructions 15
3.3.3.2 Save symmetric key blob to the file 15
3.3.3.3 Detailed Instructions 16
3.3.3.4 Testing/Error Reporting 17
4 Known Issues 17
4.1 OpenSSL Crypto provider multiclient support 17
5 FAQs 18
6 Troubleshooting/Tips and Tricks 18
6.1 OpenSSL Command Line Utility (s_client/s_server) 18
6.2 Premaster Secret 18
6.3 Key blob file structure 19
7 References 20
7.1 Abbreviations/Acronyms/Definitions 20
7.2 Platform Documentation 20
7.3 Website links 20
8 **Revision History 21**

© Continental Automotive GmbH 2019
Confidential and Proprietary

Sensitivity: Internal & Restricted

1 Introduction

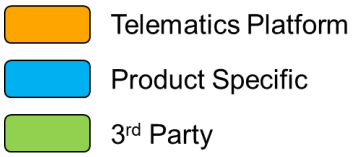
This document provides usage information for security features of the Telematics Platform.

1.1 Notational Conventions

Ctrl+x	*A sequence with a '+' indicates that you must press and hold one key or pointing device button while pressing another key or pointing device button*
[ ]	*Optional elements in a command*
…..	*Indicates one of the following possibilities:* • *Additional optional arguments in a statement have been omitted* • *The preceding item or items can be repeated one or more times* • *Additional parameters, values, or other information can be entered*
-	*Line continuation: indicates that a command line or code line continues on the following line when at the end of a line*
Italic text	*Indicates important information, complete titles of manuals or variables*
`Mono-spaced`	*Example code or interactive screen display/output*
Numbers	*All numbers are assumed to be decimal unless otherwise noted. Non decimal radixes – binary, octal or hexadecimal – are explicitly indicated*
Underline blue text	*A link to an external reference (e.g., website, document)*

`Modules`
`|-> section`
`|-> item`
	A link to a section or item in the Telematics Platform html interface specification (Doxygen output from platform code headers)

Diagram Legend



2 Setup

2.1 Tools/Equipment Needed

2.2 Setup Instructions

2.3 Setup Troubleshooting

3 Features/Components

3.1 How to setup password through persistency

3.1.1 Overview

Platform uses pluggable authentication modules (PAM) to retrieve console login (/bin/sh) password from persistent secure storage. Platform provides a framework for products to setup unique passwords for different users (root, factory etc.).

3.1.1.1 Platform Content Overview

Platform provides a secure persistent key-value resource to store console login passwords.

© Continental Automotive GmbH 2019
Confidential and Proprietary

Sensitivity: Internal & Restricted

The key-value resource is built into persistency image.

The below is an example of key-value resource where user name is “root”.

<resource>

<res_type>key-value**</res_type>**

<name>root**</name>**

<application>pam**</application>**

<storage_type>local**</storage_type>**

<permission>ro**</permission>**

<policy>secure**</policy>**

<user>root**</user>**

<group>root**</group>**

<data_type>string**</data_type>**

<default>*d417ccba6f5707e1a45c7c74662334bf$7982B25165C19BE534C0277815D39AEF668453F7776F9A3124A17AE6686C6EB7***</default>**

<default_size>97**</default_size>**

<max_data_size>128**</max_data_size>**

</resource>

Platform follows below steps to generate persistent key-value resource for console login password –

- Generate 32 bytes SALT as a random number.
- Generate SHA256 hash of the given password and SALT value
- Persistent key-value resource uses the hash value in combination with the salt value separated by a delimiter, specifically “SALT+delimiter+HASH” , where delimiter value used as **$**
- Persistency uses secure storage to encrypt “SALT+delimiter+HASH” to store in persistent database.

Platform’s persistency feature provides a framework through which products are expected to override the platform provided console login passwords.

Products are required to change the default console login passwords provided by platform.

3.1.1.2 Product Expectations

Console login passwords (respective to the user “root”, “factory” etc.) will be unique for each product.

Product must generate its own console login passwords and follow below steps to create a key-value pair resource which gets securely stored in persistent database.

Example – Considering that the product’s console login password is **“foobar”** for user **“root”.**

Generate SALT

openssl rand -hex 16

```
uidq8938@dpv7071u:~$
uidq8938@dpv7071u:~$ openssl rand -hex 16
b7639384bd2f0db9b9b7c4921110e5f0
uidq8938@dpv7071u:~$
```

Generate password HASH

© Continental Automotive GmbH 2019
Confidential and Proprietary

Sensitivity: Internal & Restricted

```
echo -n <PASSWORD+SALT> | sha256sum | tr /a-z/ /A-Z/
```

```
uidq8938@dpv7071u:~$
uidq8938@dpv7071u:~$ echo -n foobarb7639384bd2f0db9b9b7c4921110e5f0 | sha256sum | tr /a-z/ /A-Z/
2E9137D84E9AC180C1835E196F1B964D5A6A1768DCD42E000DBEB473AAEE99A8  -
```

The above mentioned steps are executed in host environment.
Concatenate the generated SALT, password HASH and delimiter ($) to add at "**default**" section of key-value resource (as below)

<resource>

<res_type>key-value**</res_type>**

<name>root**</name>**

<application>pam**</application**>

<storage_type>local**</storage_type>**

<permission>ro**</permission>**

<policy>secure**</policy>**

<user>root**</user>**

<group>root**</group>**

<data_type>string**</data_type>**

<default>b7639384bd2f0db9b9b7c4921110e5f0$2E9137D84E9AC180C1835E196F1B964D5A6A1768DCD42E000DBEB473AAEE99A8**</default>**

<default_size>97**</default_size>**

<max_data_size>128**</max_data_size>**

</resource>

The key-value resource attributes are described as below –

- **res_type:** key-value, A persistent database resource (Do not change this value)
- **name**: user name , mentioned in /etc/passwd
- **application**: *pam (*Do not change this value)
- **storage_type**: *local*, the data stored specific to application (Do not change this value)
- **permission**: *ro*, Read only data (Do not change this value)
- **policy**: *secure*, to store in encrypted format (Do not change this value)
- **user**: root (Do not change this value)
- **group**: root (Do not change this value)
- **data_type**: *string* (Do not change this value)
- **default**: the password HASH, stored in the format "SALT+delimiter+HASH"
- **default_size**: *97*, as the SALT is 32 byte chars, HASH is 64 and the delimiter 1, <64+32+1>
- **max_data_size**: *128*, as that is kept as the default max length (Do not exceed the value 128)

For more details on key-value resource attributes, refer <**Persistence User Guide**>.

To set product specific console login passwords, Products must override respective key-value resource in a persistent database. For detailed steps refer <**Persistence User Guide**>.

Products may disable console login with root privilege by deleting console login passwords from persistent database. Details can be found in "Detailed Instructions"

© Continental Automotive GmbH 2019
Confidential and Proprietary

Sensitivity: Internal & Restricted

3.1.1.3 Extending Platform Functionality

Not Applicable

3.1.2 Setup

3.1.2.1 Tools/Equipment Needed

Not Applicable

3.1.2.2 Setup Instructions

Persistency image has to be installed first before the login prompt would be available for the user.

Detailed instructions on persistency image installation can be found at <**Persistence User Guide**>.

3.1.2.3 Setup Troubleshooting

Not Applicable

3.1.3 Usage

3.1.3.1 Basic Instructions

Not Applicable

3.1.3.2 Detailed Instructions

To delete product's console login passwords from persistency database, follow the below steps –

- Install persistency (steps as mentioned in "Setup Instructions")
- In factory mode, use "persistent_key_test" tool to delete product specific console login passwords. Invoke command at target - `persistent_key_test delete pam 0xffffffff root`

The console login passwords must be deleted with every persistency (re)installation if and only if products are required to delete console login passwords from persistent database.

3.1.3.3 Testing/Error Reporting

Not Applicable

3.2 How to sandbox an application

3.2.1 Overview

Sandbox is a security mechanism for separating running programs, usually in an effort to prevent system failures or software vulnerabilities from spreading. It is often used to execute untested or untrusted programs or code, possibly from unverified or untrusted third parties, suppliers, users or websites, without risking harm to the host machine or operating system. A sandbox typically provides a tightly controlled set of resources for guest programs to run in.

3.2.1.1 Platform Content Overview

To sandbox product applications, Platform has provided a framework to configure DAC (discretionary access control) and D-Bus security policies. Platform has configured its services to use non-root and enforces D-Bus security rule.

© Continental Automotive GmbH 2019
Confidential and Proprietary

Sensitivity: Internal & Restricted

Platform has provided the following user and group range for all applications:

Security Level	User ID (Min)	User ID (Max)	Group ID (Min	Group ID (Max)
Platform	2910	2999	2910	2999
Product and Third Party	10001	12000	10001	12000

Platform does not provide support for cgroups configuration. Product application is responsible for cgroups configuration via systemd.

Platform recommends products to

- Use apparmor to scan the application's capability needed and resources used.
- Create a non-root user and group to secure the application.
- Add the created user to the proper platform group so it can access platform services.
- Grant non-root application's partial privilege by enabling required capabilities(setcap)
- Add D-Bus security policy rule for the application.
- Add proper access permission for shared resources created by application.

3.2.1.2 Product Expectations

Product application must specify respective user and group ID in systemd unit file if systemd is used to start the application. See an example systemd unit file having user and group ID defined at "Setup instructions".

Product applications must add its own D-Bus security policy based on its need to access platform services. Any Product application as a D-Bus client needs to use the Platform service has to add its user to the corresponding Platform service group. And Product application as a D-Bus service has to design its own D-Bus policy rule to limit its service. See an example of D-Bus security policy and service file at "Setup instructions".

For dynamically created resources by applications, Products may create and/or maintain udev rules to grant permission to device (/dev) and system (/sys/) files. Products may set appropriate permission for sharing with other applications. Products may use systemd started script to configure pre-known resource permission.

3.2.1.3 Extending Platform Functionality

Not Applicable

3.2.2 Setup

3.2.2.1 Tools/Equipment Needed

Not Applicable

3.2.2.2 Setup Instructions

© Continental Automotive GmbH 2019
Confidential and Proprietary

Sensitivity: Internal & Restricted

Example of systemd unit file where application/process will be executing with User as "**power**", primary Group as **"power"** and SupplementaryGroups as "**otp**".

(Reference file location - /lib/systemd/system/lifecyclemgr.service)

```
[Unit]
Description=Lifecycle Manager Service
Requires=dbus.service
After=dbus.service conti-nadif.service vcs.service
[Service]
User=power
Group=power
SupplementaryGroups=otp
Type=dbus
BusName=com.contiautomotive.Lifecycle
ExecStartPre=/bin/rm -f /var/run/lifecyclemgr/lifecycle-mgrd-lock
ExecStart=CONTI_IPR_DIR/bin/lifecycle-mgrd
Restart=always
RestartSec=0
WatchdogSec=15
[Install]
WantedBy=multi-user.target
```

Examples of D-Bus Security policies can be found at - /etc/dbus-1/

3.2.2.3 Setup Troubleshooting

Not Applicable

3.2.3 Usage

3.2.3.1 Basic Instructions

Not Applicable

3.2.3.2 Detailed Instructions

Not Applicable

3.2.3.3 Testing/Error Reporting

Not Applicable

© Continental Automotive GmbH 2019
Confidential and Proprietary

Sensitivity: Internal & Restricted

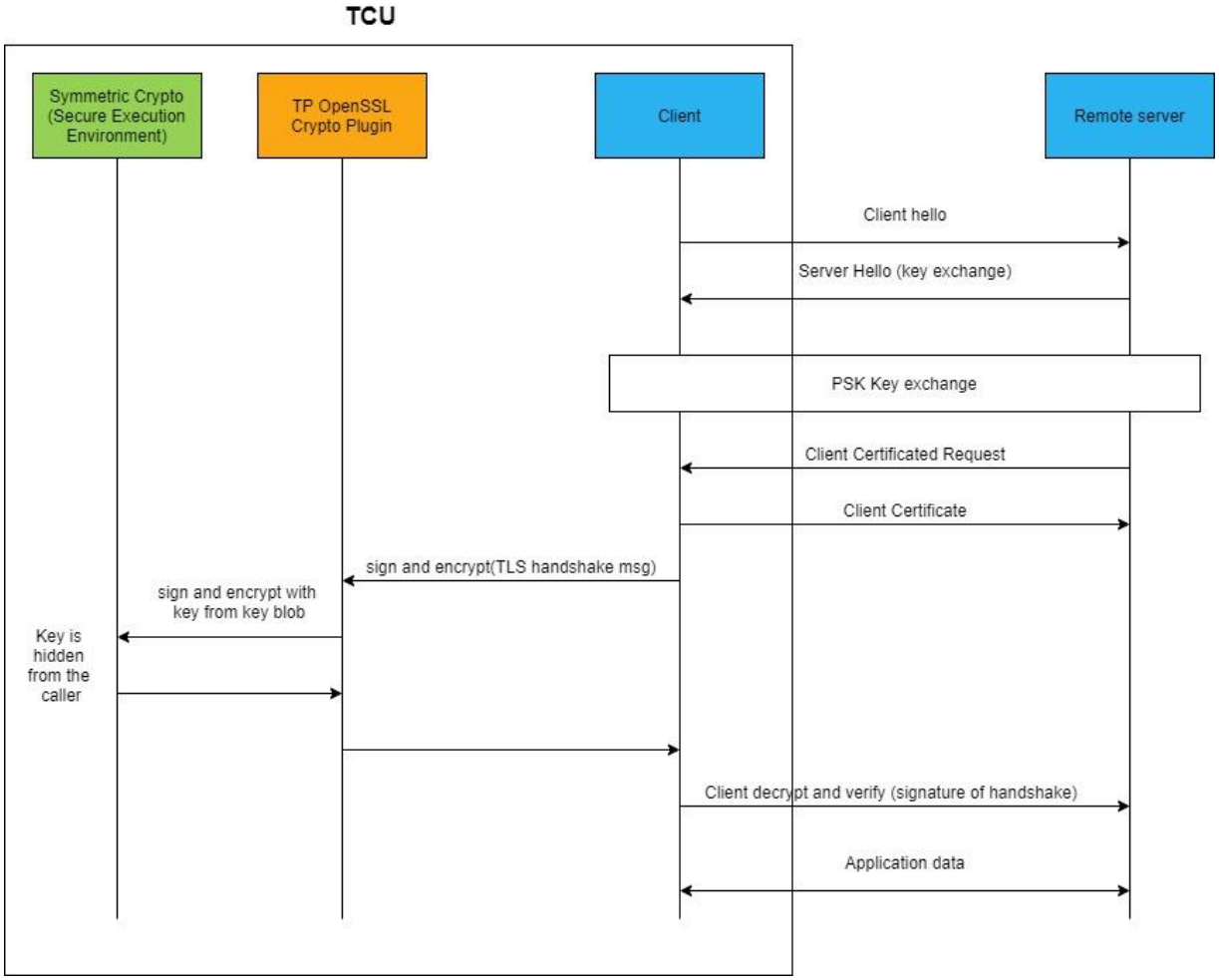
3.3 OpenSSL Crypto Provider

3.3.1 Overview

The OpenSSL Crypto Provider is an OpenSSL provider implementation that uses the Security HAL Interface to perform the AES128-GCM crypto and sign/verify operations and HMAC SHA256 sign/verify operations, keeping the private key completely hidden from the end-user. Keeping the private key and symmetric key hidden is critical, as it is used to uniquely identify the TCU. If a hacker obtains the key, they could masquerade as the owner of the key.

Any application that uses the OpenSSL library can use the OpenSSL Crypto Provider to perform the HMAC operations and leverage the advanced security features of the Telematics Platform to protect the PSK keys.

Following is an example of how the OpenSSL Crypto Provider is used to enhance the security of the sign operation by hiding the PSK key.



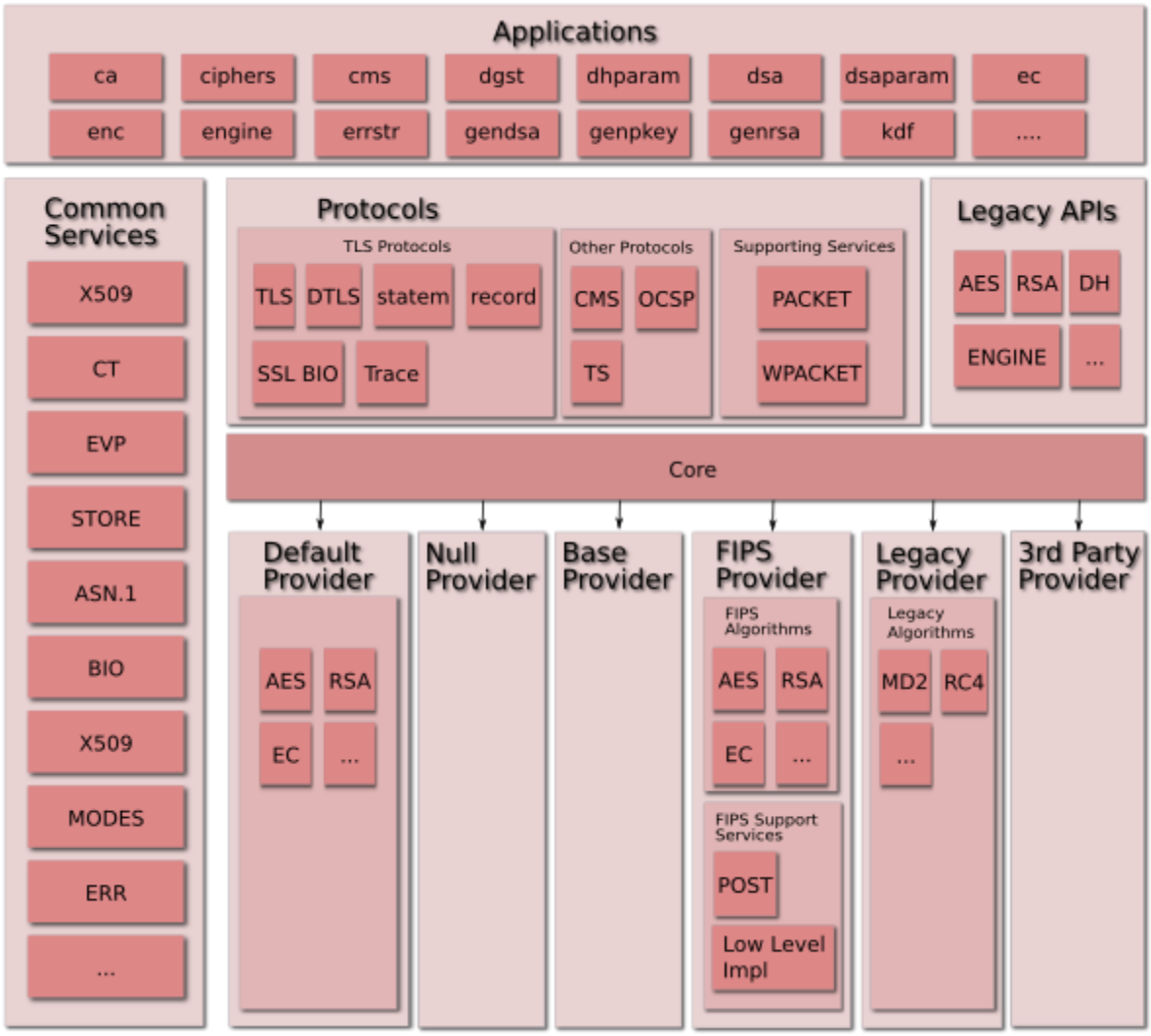
NOTE: The client can be any program using OpenSSL, such as libcurl, or a custom application using OpenSSL.

© Continental Automotive GmbH 2019
Confidential and Proprietary

Sensitivity: Internal & Restricted

3.3.1.1 Platform Content Overview

The OpenSSL Crypto Provider integrates with the OpenSSL software stack in the following way:



NOTES: the existence of a component in the diagram does not indicate that the component is a public API or intended for end-user direct access or usage.

The components shown here are as follows:

- Applications: Command line applications, e.g. ca, ciphers, cms, dgst etc
- Protocols: Provides capabilities for communicating between endpoints according to standard protocols
    - TLS Protocols: An implementation of all supported TLS/DTLS protocols and supporting infrastructure such as:

© Continental Automotive GmbH 2019
Confidential and Proprietary

Sensitivity: Internal & Restricted

        - SSL BIO: A BIO for communication using TLS
        - Statem: The TLS state machine
        - Record: The TLS record layer
    - Other Protocols
        - CMS: An implementation of the Cryptographic Message Syntax standard
        - OCSP: An implementation of Online Certificate Status Protocol
        - TS: An implementation of the Timestamp Protocol
    - Supporting Services: Components specifically designed to support the implementation of protocol code
        - Packet: Internal component for reading protocol messages
        - Wpacket: Internal component for writing protocol messages
- Core: This is a fundamental component that connects requests for a service (such as encryption) to a provider of that service. It implements the ability for providers to register their services along with the properties of those services. It also provides the ability to locate a service given a set of properties that the service must fulfil. For example, properties of an encryption service might include “aead”, “aes-gcm”, “fips”, “security-bits=128”, etc.
- Default Provider: Implements a set of default services that are registered with the Core.
    - Supporting Services
        - Low Level Implementations: This is the set of components that actually implement the cryptographic primitives.
- FIPS Provider: Implements a set of services that are FIPS validated and made available to the Core. This includes the following supporting services:
    - POST: Power On Self Test
    - KAT: Known Answer Tests
    - Integrity Check
    - Low Level Implementations: This is the set of components that actually implement the cryptographic primitives (to meet the FIPS-mandated self-contained requirement).
- Legacy Provider: Provides implementations of older algorithms that will be exposed via EVP-level APIs.
- Third-Party Provider: Not part of the OpenSSL distribution. Third Parties may implement their own providers.
- Common Services: these form the building blocks usable by applications and providers. (e.g. BIO, X509, SECMEM, ASN1, etc).
- Legacy APIs. The “low-level” APIs. The “legacy” here refers to the API, not the algorithm itself. For example, AES is not a legacy algorithm but it has a legacy API (e.g. AES_encrypt).

The OpenSSL Crypto Providercurrently supports the following operation:

Sign/Verify by MAC with the PSK size up to 256 and sign with HMAC based on SHA256 with the PSK size up to 256

The plugins use the Security HAL Interface to perform the actual PSK sign operation. This allows for the PSK key used to perform the signature or crypto operation to be kept hidden (encrypted) from the caller, thus increasing the overall security of the operation:

- The PSK key is hidden from the high-level operating system, encrypted with a symmetric key inaccessible to the user.
- A trusted application running in the secure execution environment provided by the underlying hardware will perform the PSK sign operation

© Continental Automotive GmbH 2019
Confidential and Proprietary

Sensitivity: Internal & Restricted

The intended use case for the PSK sign/crypto operation using the Security HAL Interface is to secure the key used during the authentication step of the TLS handshake.

3.3.1.2 Product Expectations

PSK key must be created and imported into the secure execution environment. The PSK key converted to the premaster secret[6] and imported into the secure execution environment with the Security HAL Interface.

When the PSK key import is completed client receives a key blob and shall pass it to OpenSSL via file in client application for further use (see section 3.3.3.2 Save symmetric key blob to the file).

3.3.1.3 Extending Platform Functionality

Not applicable.

3.3.2 Setup

This section describes the steps involved to setup the environment to use the Security HAL Interface. References to Security APIs are types can be found in this section, for more details on these APIs and types see the Doxygen output (HAL Security Interface) for the Security Interface APIs:

API/Type Description	Doxygen Path
API to imports the symmetric key	Modules |-> Hardware Abstraction Layer |-> HAL Library |-> Security Interface |-> **tpHAL_importSymmetricKey()**
API to delete the symmetric key	Modules |-> Hardware Abstraction Layer |-> HAL Library |-> Security Interface |-> **tpHAL_deleteSymmetricKey()**
PSK key blob type (used by Security API to store and hide PSK keys)	Modules |-> Hardware Abstraction Layer |-> HAL Library |-> Security Interface |-> **tpHAL_keyData_t**
Encrypts and sign the plain text with key blob and generate ciphertext and MAC authentication tag over the encrypted data.	Modules |-> Hardware Abstraction Layer |-> HAL Library |-> Security Interface |-> **tpHAL_encryptDataAES_GCM()**
Decrypts the ciphertext using the key blob and verifies ciphertext and/or AAD with MAC authentication tag.	Modules |-> Hardware Abstraction Layer |-> HAL Library |-> Security Interface |-> **tpHAL_decryptDataAES_GCM()**

© Continental Automotive GmbH 2019
Confidential and Proprietary

Sensitivity: Internal & Restricted

Signs data with the HMAC algorithm	Modules |-> Hardware Abstraction Layer |-> HAL Library |-> Security Interface |-> **tpHAL_signDataHMAC()**
Verifies data with the HMAC algorithm	Modules |-> Hardware Abstraction Layer |-> HAL Library |-> Security Interface |-> **tpHAL_verifyDataHMAC()**

3.3.2.1 Tools/Equipment Needed

Not applicable.

3.3.2.2 Setup PSK Key Instruction

This section details the steps to create a PSK key, and to store the key using the Security HAL Interface.

3.3.2.2.1 Importing PSK key via the Secure HAL Interface

The PSK key imports to the target TCU using the **tpHAL_importSymmetricKey()** API. The externally created PSK key must be converted to the premaster secret (see section 6.4 for an example how to create premaster secret) and then passed as arguments to the API:

```
tpHAL_importSymmetricKey(tpHAL_keyData_t* key_blob,
const tpHAL_keyType_t key_type,
const tpHAL_keyData_t *key_data);
```

Parameters:

[out] **key_blob** Pointer to an opaque key blob. The caller assumes ownership of the contained key material
On return, this is filled with the imported key obfuscated blob

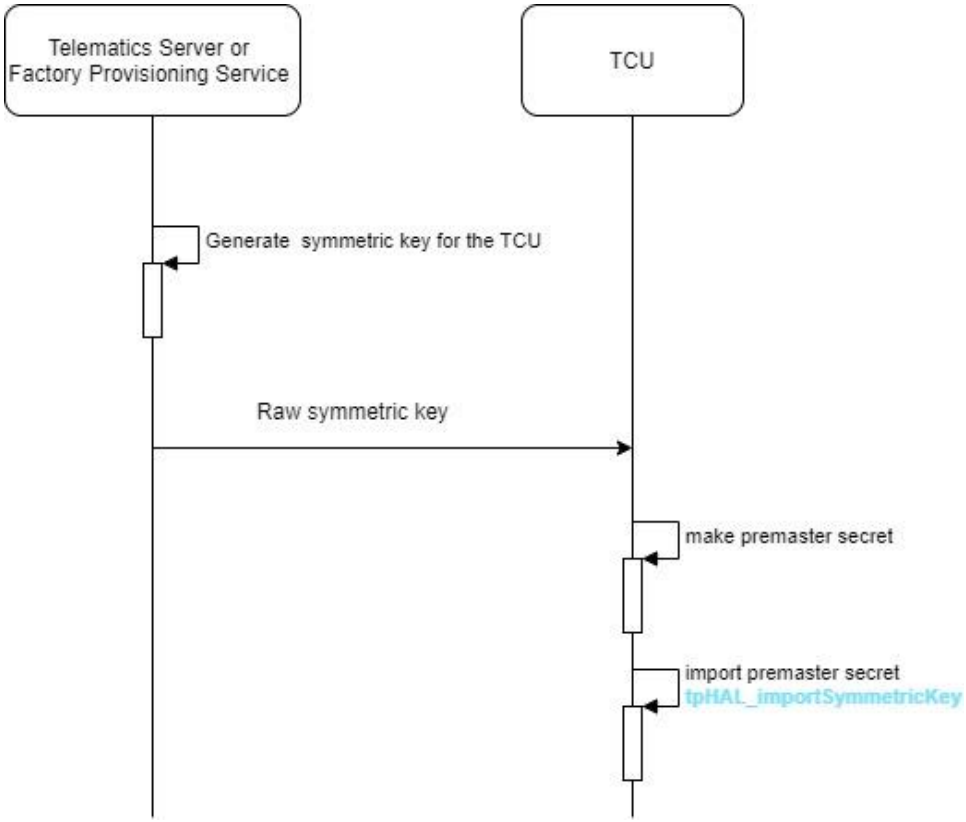
[in] **key_type** Type of the key to import

[in] **key_data** Raw key data provided in one of the supported formats

The following diagram depicts a typical symmetric key import procedure

© Continental Automotive GmbH 2019
Confidential and Proprietary

Sensitivity: Internal & Restricted



3.3.2.3 Setup Troubleshooting

A possible source of errors is the HAL setup. If the Security HAL APIs are failing, this may point to a setup issue. In this case, HAL setup needs to be performed before the Security HAL APIs can be used. Refer to your HAL vendor for setup/install information

3.3.3 Usage

3.3.3.1 Basic Instructions

This section describes how to save key blob into the file and how to use the OpenSSL Crypto provider

3.3.3.2 Save symmetric key blob to the file

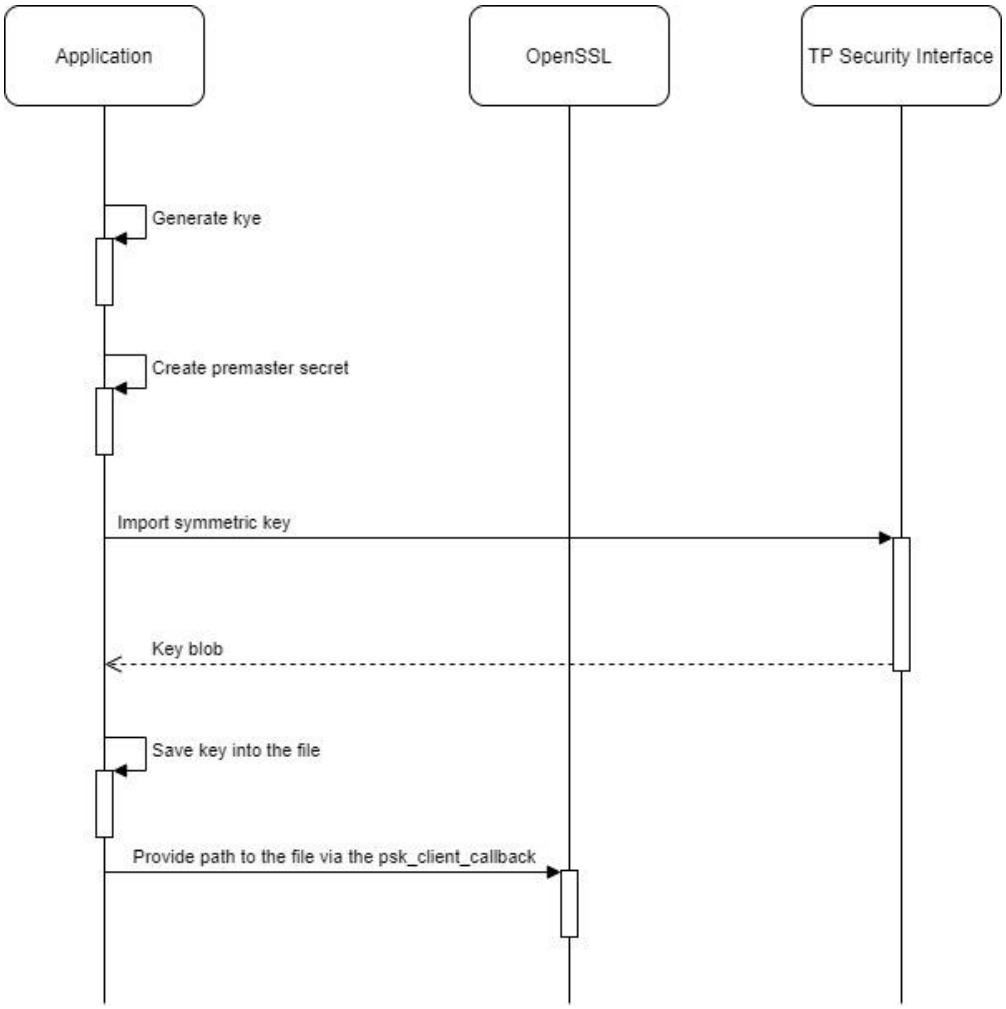
PSK key should be converted to the premaster secret then the premaster secret should be imported into the secure execution environment.When the key is imported, and the key blob is obtained, key blob shall be saved into the file system. Path to the file should be used instead of the PSK key and provided from the client side through the **SSL_psk_client_cb_func**.

The root directory for the key blob's file storage is '/data'. For file structure definition please refer to section 6.3 Key blob file structure.

For more details check the OpenSSL man page [18].

© Continental Automotive GmbH 2019
Confidential and Proprietary

Sensitivity: Internal & Restricted



3.3.3.3 Detailed Instructions

Any software application that uses the OpenSSL API [1] to configure and use cryptographic operations can use the OpenSSL Crypto provider. The following sections describe how to use the OpenSSL API to use the OpenSSL Crypto provider, and also details of how to integrate with popular software applications that perform PSK sign operations.

In OpenSSL 3.0, the concept of providers was introduced to allow users to extend OpenSSL with new cryptographic algorithms. In the application code, need to explicitly load the provider before it can be used. This is typically done using the OPENSSL_init_crypto() function with the OPENSSL_INIT_LOAD_CONFIG flag, which loads configuration files and initializes the OpenSSL library. Here's an example:

Copy code

```
#include <openssl/ssl.h>

int main() {
   SSL_library_init();
```


© Continental Automotive GmbH 2019
Confidential and Proprietary

Sensitivity: Internal & Restricted

```
    OPENSSL_init_ssl(OPENSSL_INIT_LOAD_CONFIG, NULL);

    // Load the provider
    const char *provider_path = "/path/to/provider.so"; // Replace this with the actual path to your provider
    OSSL_PROVIDER *provider = OSSL_PROVIDER_load(NULL, provider_path);

    if (provider == NULL) {
        fprintf(stderr, "Failed to load provider\n");
        exit(EXIT_FAILURE);
    }

    // Use the provider for TLS client operations
    // For example, set it as the default provider for TLS client
    SSL_CTX *ctx = SSL_CTX_new(TLS_client_method());
    SSL_CTX_set_default_provider(ctx, provider);

    // Now you can create SSL connections using this SSL_CTX

    // Clean up
    OSSL_PROVIDER_unload(provider);
    SSL_CTX_free(ctx);

    return 0;
}
```

Replace "/path/to/provider.so" with the actual path to your provider's shared library file.

Once the provider is loaded and configured, you can use it for TLS client operations. This typically involves setting it as the default provider for the SSL context used for client connections.

Remember to handle errors properly and ensure that the provider you're loading is compatible with your TLS client requirements.

3.3.3.4 Testing/Error Reporting

Not applicable.

4 Known Issues

4.1 OpenSSL Crypto provider multiclient support

The AES128-GCM/HMAC encryption/sign provider does not support a concurrent multiclient environment. As such, OpenSSL is limited to use by one client at a time when the PSK/HMAC plugin is loaded.

© Continental Automotive GmbH 2019
Confidential and Proprietary

Sensitivity: Internal & Restricted

5 FAQs

Not applicable.

6 Troubleshooting/Tips and Tricks

6.1 OpenSSL Command Line Utility (s_client/s_server)

Following is an example of how to use the OpenSSL s_client [3], and s_server [4] commands to perform a TLS handshake from a TCU using the OpenSSL Crypto provider. Note that these commands will work only if the client and server certificates are setup properly, and the steps for that are not included here.

PSK-AES128-GCM-SHA256:

- Run with a remote server

  Start the server on a remote host accessible by the target:

```
openssl s_server -cipher PSK-AES128-GCM-SHA256 -nocert -psk 123456 -tls1_2
```

  Start the client on the target:

```
openssl s_client -connect 176.36.118.197:4433 -cipher PSK-AES128-GCM-SHA256 -psk 123456 -tls1_2 -provider-path /data -provider libprovider-aes-gcm-hmac -provider default
```

- Run with local server

  Open adb shell and start the server on the target:

```
openssl s_server -cipher PSK-AES128-GCM-SHA256 -nocert -psk 123456 -tls1_2
```

  Open separate adb shell instance and start the client on the target:

```
openssl s_client -cipher PSK-AES128-GCM-SHA256 -psk 123456 -tls1_2 -provider-path /data -provider libprovider-aes-gcm-hmac -provider default
```

PSK-NULL-SHA256:

- Run with a remote server

  Start the server on a remote host accessible by the target:

```
openssl s_server -cipher PSK-NULL-SHA256 -nocert -psk 123456 -tls1_2
```

  Start the client on the target:

```
openssl s_client -connect 176.36.118.197:4433 -cipher PSK-NULL-SHA256 -psk 123456 -tls1_2 -provider-path /data -provider libprovider-aes-gcm-hmac -provider default
```

- Run with local server

  Open adb shell and start the server on the target:

```
openssl s_server -cipher PSK-NULL-SHA256 -nocert -psk 123456 -tls1_2
```

  Open separate adb shell instance and start the client on the target:

```
openssl s_client -cipher PSK-NULL-SHA256 -psk 123456 -tls1_2 -provider-path /data -provider libprovider-aes-gcm-hmac -provider default
```

6.2 Premaster Secret

© Continental Automotive GmbH 2019
Confidential and Proprietary

Sensitivity: Internal & Restricted

The premaster secret is formed as follows: if the PSK is N octets long, concatenate a uint16 with the value N, N zero octets, a second uint16 with the value N, and the PSK itself.

Note: PSK-AES128-GCM-SHA256 and SK-NULL-SHA256 cipher suites share the same general structure for the premaster secret:

```
struct {
        other_secret<0..2^16-1>;
       psk<0..2^16-1>;
};
```

Here "other_secret" either is zeroes.

The following code is an example of how to make a premaster secret:

```
tpHAL_return_t make_premaster_secret(tpHAL_keyData_t* input_key, tpHAL_keyData_t* out_key)
{
    out_key->data_length = input_key->data_length*2+SIZE_FIELDS;
    uint8_t* pre_master_storage = malloc(out_key->data_length);
    if (pre_master_storage == NULL) {
        return E_HAL_RET_GENERIC_ERROR;
    }
    memset(pre_master_storage, 0, out_key->data_length);
    pre_master_storage[1] = input_key->data_length;
    pre_master_storage[input_key->data_length+3] = input_key->data_length;
    memcpy((void*)(pre_master_storage+input_key->data_length+SIZE_FIELDS),
    input_key->data, input_key->data_length);
    out_key->data = pre_master_storage;
    return E_HAL_RET_SUCCESS;
}
```

6.3 Key blob file structure

0 … 3 bites	4 bites … end of the file
key blob size	**key blob**

© Continental Automotive GmbH 2019
Confidential and Proprietary

Sensitivity: Internal & Restricted

7 References

7.1 Abbreviations/Acronyms/Definitions

7.2 Platform Documentation

TP_102G-00001: Telematics Platform Glossary

7.3 Website links

Ref	Name	Location
1	OpenSSL EVP	https://wiki.openssl.org/index.php/EVP
2	OpenSSL provider	/docs/man3.0/man7/provider.html (openssl.org)
3	OpenSSL s_client	https://www.openssl.org/docs/manmaster/man1/s_client.html
4	OpenSSL s_server	https://www.openssl.org/docs/manmaster/man1/s_server.html
5	Libcurl library	https://curl.haxx.se/libcurl/
6	PSK Key Exchange Algorithm	https://tools.ietf.org/html/rfc4279#section-2
7	AES Galois Counter Mode	https://tools.ietf.org/html/rfc5288
8	Ciphersuites with NULL Encryption	https://tools.ietf.org/html/rfc4785
9	Linux PAM	http://www.linux-pam.org/
10	Sandboxing	https://en.wikipedia.org/wiki/Sandbox_(computer_security)
11	dbus-daemon	https://dbus.freedesktop.org/doc/dbus-daemon.1.html
12	Linux cgroups	http://man7.org/linux/man-pages/man7/cgroups.7.html
13	libcgroup rules	https://linux.die.net/man/5/cgrules.conf
14	libcgroup configuration	https://linux.die.net/man/5/cgconfig.conf
15	OpenSSL Wiki	https://wiki.openssl.org/index.php/Main_Page
16	OpenSSL config	https://www.openssl.org/docs/manmaster/man5/config.html
17	HMAC	https://tools.ietf.org/html/rfc2104
18	OpenSSL set_psk_client_callback	https://www.openssl.org/docs/man1.1.1/man3/SSL_CTX_set_psk_client_callback.html
19	Description of systemd directives for cgroups	https://www.freedesktop.org/software/systemd/man/systemd.resource-control.html

© Continental Automotive GmbH 2019
Confidential and Proprietary

Sensitivity: Internal & Restricted

8 Revision History

Version	Date	Description
01.00	Aug 15, 2019	Initial Release
02.00	Aug 21, 2020	Linux User and Group Identity Range
03.00	Jan 18, 2021	Added PSK cryptoplugin
04.00	Mar 01, 2021	Cgroup configurations removed
== ConMod-specific content from here ==		
05.00	Apr 30, 2024	Update OpenSSL 1.x Crypto engine to OpenSSL 3.x Crypto provider
06.00	Jul 3, 2024	Minor changes and corrections

© Continental Automotive GmbH 2019
Confidential and Proprietary

Sensitivity: Internal & Restricted